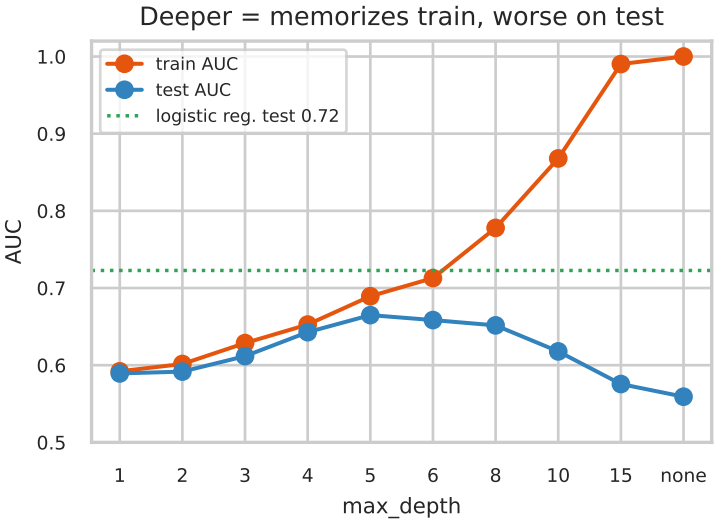
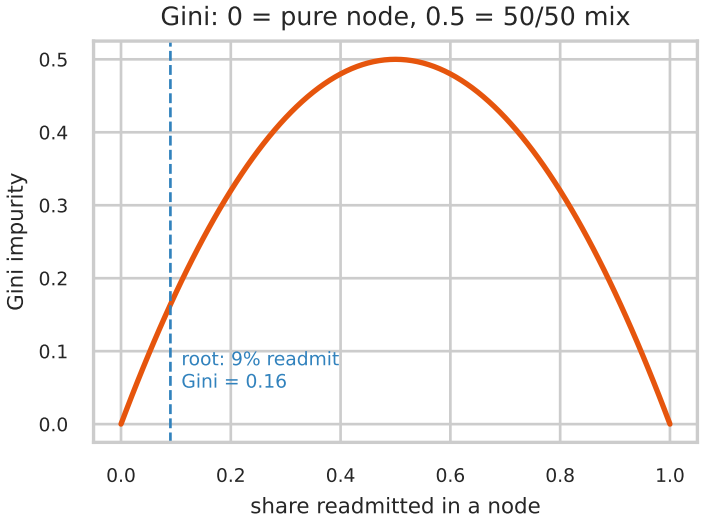
Day 15 Cheat Sheet: Decision Trees

Week 3 starts: if/then splits, depth, overfitting, and cross-validation in scikit-learn

TERM	WHAT IT MEANS	CODE
Decision tree	a stack of yes/no questions ending in a risk group	DecisionTreeClassifier()
Split	question like "prior_admits <= 0.5?" on one feature	chosen to make children purer
Gini impurity	2p(1-p) for 2 classes: 0 = pure, 0.5 = 50/50	criterion="gini" (default)
Leaf	final group; its predicted risk = share readmitted	tree.predict_proba(X)[:, 1]
max_depth	how many questions deep the tree can go	max_depth=4
min_samples_leaf	smallest allowed leaf (stops tiny, noisy groups)	min_samples_leaf=50
Overfitting	great on train data, worse on new patients	compare train vs test AUC
Cross-validation	5 train/validate rounds inside the training set	GridSearchCV(..., cv=5)
Feature importance	how much each feature cut impurity (not causation)	tree.feature_importances_
Readable rules	print the tree as if/then text	export_text(tree, feature_names=...)





THIS DATA (same simulated day 14 data, seed 14): 4,000 discharges, 9.0% readmitted, 3,000 train / 1,000 test
Depth-3 readable tree: test AUC 0.611 | unlimited depth: 497 leaves, train AUC 1.00, test AUC 0.56
Best by test set: depth 5 (0.665), but you pick with CV on train only: {'max_depth': 4, 'min_samples_leaf': 50}
Tuned tree: CV AUC 0.614, test AUC 0.649 vs day 14 logistic regression 0.723
Importance: prior_admits 0.39, los_days 0.25, age 0.19, n_meds 0.13, followup_visit 0.00

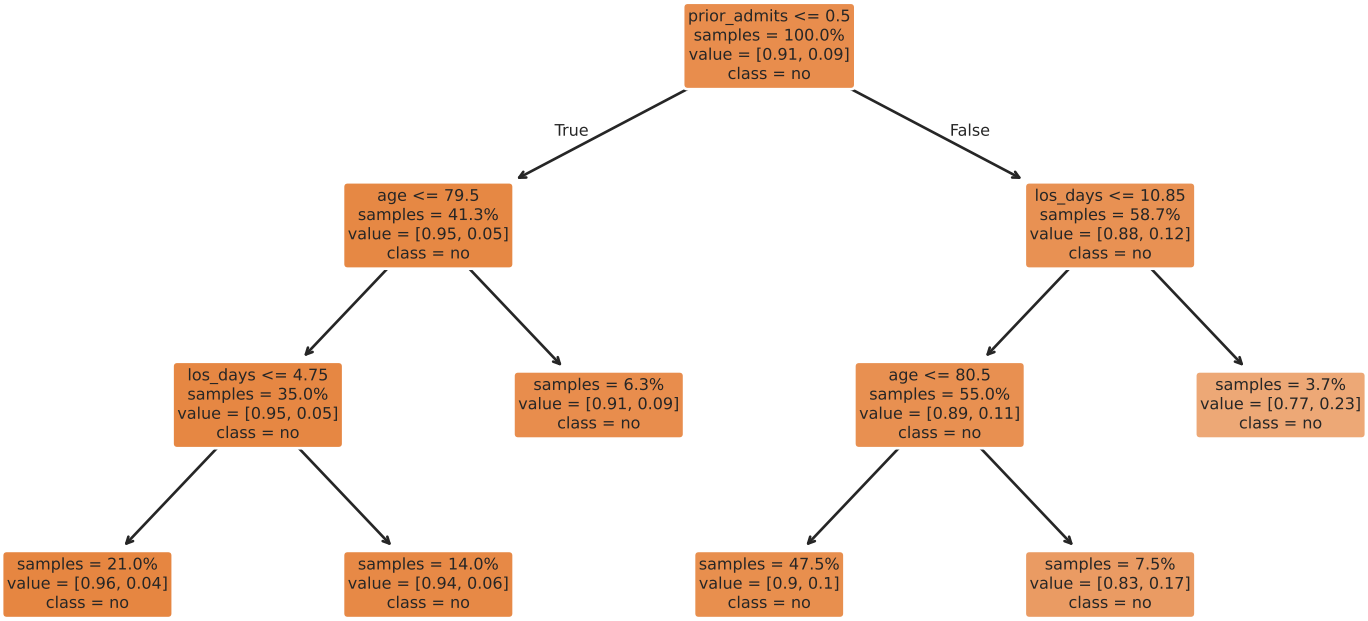
HABITS THAT KEEP YOU HONEST
- Always print train AUC next to test AUC: a big gap is overfitting
- Tune depth and leaf size with cross-validation on the training set, never on the test set
- A tree is not automatically better: here a straight-line logistic model wins
- Every leaf can predict "no" at a 9% base rate; use predict_proba, not predict
- Importance shows what the tree used, not what causes readmission

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(DecisionTreeClassifier(random_state=14),
       {"max_depth": [2, 3, 4, 5, 6], "min_samples_leaf": [10, 25, 50, 100]}, scoring="roc_auc", cv=5)
p = grid.fit(X_train, y_train).best_estimator_.predict_proba(X_test)[:, 1]
```

Infographic: How a Tree Sorts 3,000 Discharges

Simulated readmissions data from day 14. Same split, new model family

Depth-3 tree (min 100 patients per leaf): value = [no, readmit] share



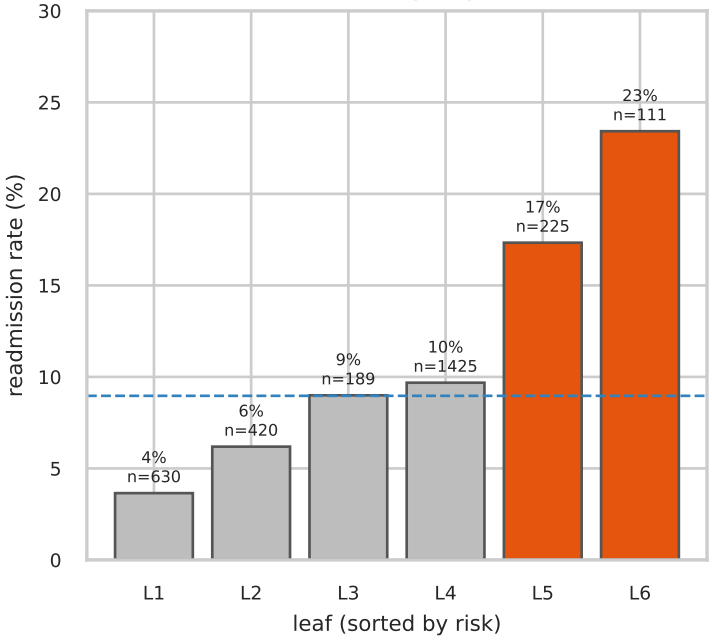
ROOT GINI
0.16
9% readmitted

READABLE TREE
AUC 0.61
6 leaves

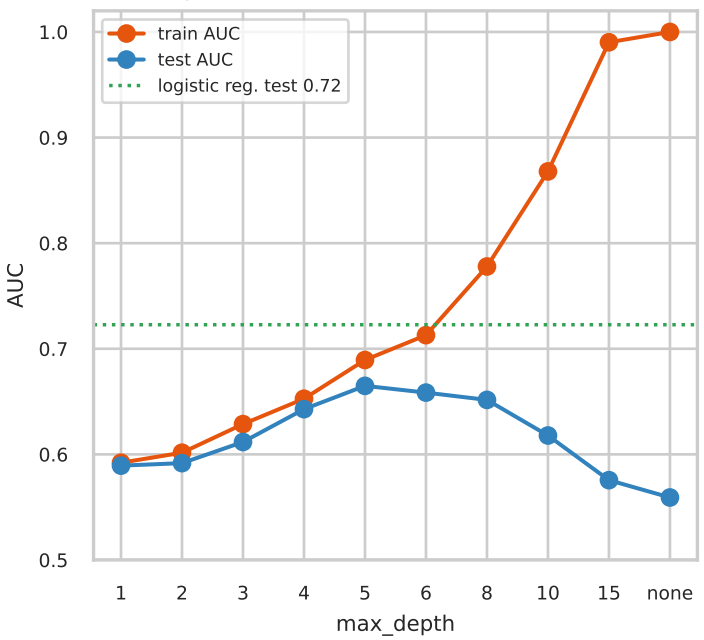
NO DEPTH LIMIT
train 1.00
test 0.56

TUNED TREE
AUC 0.65
vs logistic 0.72

Each leaf is a risk group (train set)



Deeper = memorizes train, worse on test



Read it like a care manager: the tree finds plain groups, like 'any prior admission and a stay over 10.9 days' (about 23% readmitted, n=111). Let it grow without limits and it memorizes training patients (train AUC 1.00) but falls to 0.56 on new ones. Tomorrow, random forests average many trees to fix that.

Hands-On Worksheet: ds-genai-90-days / day15

60-90 minutes | notebook: day15/decision_trees.ipynb + day15/README.md

[ ] mkdir day15, conda activate ds90, open decision_trees.ipynb

[ ] Rebuild the day 14 data and the same split (snippet 1)

[ ] Compute root Gini by hand: 2 x 0.09 x 0.91 = _____

[ ] Fit a depth-3 tree, min_samples_leaf=100; print export_text (snippet 2)

[ ] Plot it with plot_tree(filled=True, proportion=True)

[ ] Write the riskiest leaf as one plain-English rule

[ ] Depth sweep 1 to unlimited: train vs test AUC table (snippet 3)

[ ] Line chart of train vs test AUC; mark where overfitting starts

[ ] GridSearchCV on max_depth x min_samples_leaf, 5-fold (snippet 4)

[ ] Score the tuned tree once on the test set

[ ] Compare with logistic regression; plot both ROC curves (snippet 5)

[ ] Bar chart of feature_importances_; note what it does NOT mean

[ ] README: 3 charts, 4 findings, why the tree lost here

[ ] git add day15 && git commit && git push (snippet 6)

[ ] Stretch: turn the riskiest leaf into a Snowflake SQL flag (snippet 7)

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
tree3 = DecisionTreeClassifier(max_depth=3, min_samples_leaf=100, random_state=14).fit(X_train, y_train)
print(export_text(tree3, feature_names=features, show_weights=True))
for d in [1, 2, 3, 4, 5, 6, 8, 10, 15, None]:      # overfitting sweep
    t = DecisionTreeClassifier(max_depth=d, random_state=14).fit(X_train, y_train)
```

Root Gini: _____ Depth-3 AUC: _____ Best CV params: _____ CV AUC: _____

Tuned tree test AUC: _____ Logistic AUC: _____ Full-depth train / test AUC: _____ / _____

Day 15 Self-Check Quiz

1. In one sentence, what question does each split in a decision tree ask?

_____

2. A node has 9% readmitted. What is its Gini impurity? What would a pure node score?

_____

3. The unlimited tree has train AUC 1.00 and test AUC 0.56. What happened?

_____

4. Name two settings that stop a tree from overfitting.

_____

5. Why choose max_depth with cross-validation instead of the test set?

_____

6. Every leaf predicts class 0. Why, and what should you use instead?

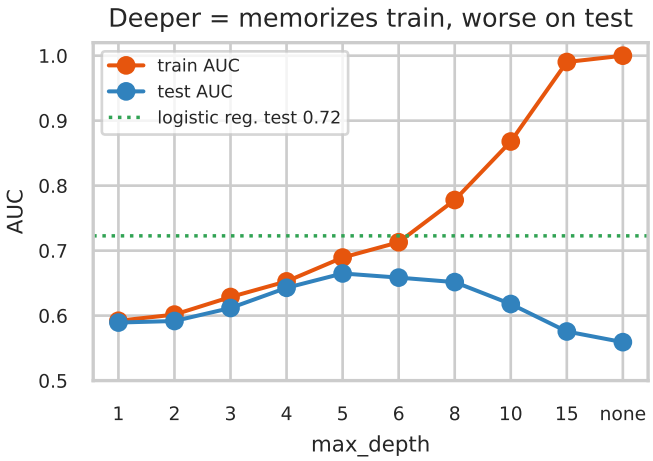
_____

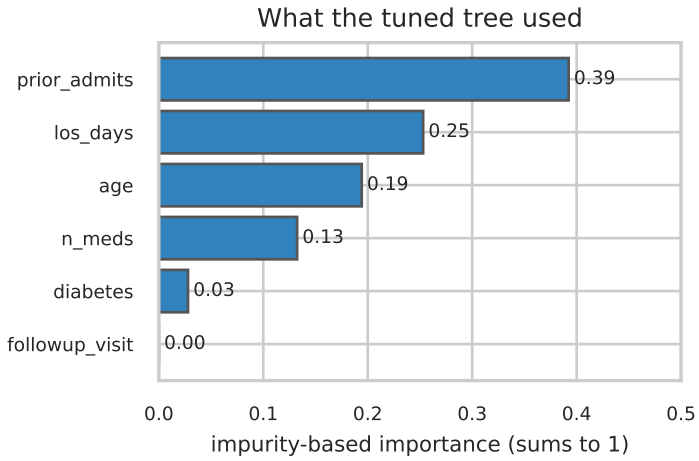
7. followup_visit has importance 0.00. Does that mean visits do not matter?

_____

8. The tuned tree (0.65) lost to logistic regression (0.72). Give one reason.

_____





Answers:
1 'is this one feature above or below a cut point?', picked so the two child groups are as pure (all readmit or all not) as possible
2 2 x 0.09 x 0.91 = 0.16; a pure node (0% or 100%) scores 0
3 overfitting: with 497 leaves it memorized the training patients, noise included, so it ranks new patients barely better than a coin
4 any two of max_depth, min_samples_leaf, min_samples_split, max_leaf_nodes, ccp_alpha (pruning)
5 if you tune on the test set it stops being a fair final exam; your reported AUC would be too optimistic
6 at a 9% base rate no leaf is above 50% readmitted, so the 0.5 cutoff says 'no' everywhere; use predict_proba risk scores and rank
7 no: the tree just never chose it; day 14 showed a real but small effect (OR 0.76). Importance is not cause and not effect size
8 the true risk here is a smooth sum of effects (we simulated it that way); a logistic model fits that shape and a tree uses coarse steps